Computing the Longest Common Subsequence of Multiple RLE Strings *

Ling-Chih Yao and Kuan-Yu Chen
Graduate Institute of Networking and Multimedia
Department of Computer Science and Information Engineering
National Taiwan University, Taipei, Taiwan 106

Abstract

In this paper, we study the problem of computing the longest common subsequence (LCS) of multiple compressed strings. The input strings are all represented by run-length encoded format. We present two algorithms for the problem. Given d strings, which are compressed into of $n_1, \ldots, n_d$ runs respectively, our algorithm computes the LCS of the d strings in $O(dnm)$ time, where $n = \sum_{i=1}^{d} n_i$ and $m = \prod_{i=1}^{d} n_i$. The second algorithm achieves $O(dn\rho|\Sigma|)$ time, where ρ denotes the number of dark blocks plus one and $|\Sigma|$ denotes the alphabet size.

1 Introduction

Longest common subsequence (LCS) serves as a well-known measure of the similarity between strings, while run-length encoding (RLE) is a classic method to compress strings. In this paper, we study a problem which lies in the intersection of the two notions, i.e. the problem of computing the similarity between compressed strings. More specifically, we consider the problem of computing longest common subsequence (LCS) of run-length encoding (RLE) strings.

Several works on computing the LCS of RLE strings have been proposed. Bunke and Csirik [4] proposed an algorithm running in $O(Nl + kM)$ time for strings of lengths M and N with l and k runs, respectively. The algorithm divides the dynamic programming table into blocks and computes only the entries on the boundaries of the blocks. Apostolico *et al.* [3] gave an algorithm running in $O(kl(k+l))$ time which relies on the notion of *forced and corner paths*. They next improved upon the time of the algorithm to $O(kl \log kl)$ time by storing all the possible forced paths in balanced binary search trees. Ann and Yang *et al.* [2] proposed an algorithm of $O(kl + \min\{p_1, p_2\})$ time, where blocks are divided into dark and light blocks and p_1 and p_2 denote the bottom and right boundaries of dark blocks.

Computing the similarity between RLE strings has been studied in more general distance settings. For example, computing the Levenshtein distance between RLE strings was explored in [5, 9]. Computing the optimal alignment between RLE strings under unrestricted scoring matrices was investigated in [6, 8].

In this paper, we study the problem of computing the LCS between RLE strings. In particular, we consider a natural generalization of the problem, i.e. computing the LCS of multiple strings. In other words, our algorithms are capable of computing the LCS between not only two RLE strings but multiple RLE strings. The paper is organized as follows. We first describe an efficient algorithm which can directly cope with compressed strings, without resorting to any decompression. Given d strings, which are compressed into $n_1, \ldots, n_d$ runs respectively, our first algorithm achieves the time complexity of $O(dnm)$ time, where $n = \sum_{i=1}^{d} n_i$ and $m = \prod_{i=1}^{d} n_i$. We next present an improved algorithm which achieves $O(dn\rho|\Sigma|)$ time, where ρ denotes the number of dark blocks plus one and $|\Sigma|$ denotes the alphabet size.

2 Preliminaries

A string S is run-length encoded (RLE) if it is described as an ordered sequence of pairs (σ, i), where σ is an alphabet symbol and i is a positive integer. Each pair corresponds to a run of S which consists of i consecutive occurrences of σ. For example, *aaabbbbccddd* can be encoded into

*This is the footnote of this paper.

$a^3b^4c^2d^3$.

2.1 Notation

Consider strings S_i for $i = 1, 2, \ldots, d$ over a finite alphabet Σ. Suppose that S_i can be encoded as an RLE string of n_i runs, denoted by $S_i(1)S_i(2)\ldots S_i(n_i)$, where $S_i(j)$ is the j-th run of string S_i. Note that we assume that the RLE string of S_i is optimally encoded, i.e. each run $S_i(j)$ is a maximal run of identical characters. We let $|S_i(j)|$ denote the run length and $symbol(S_i(j))$ denote the encoded symbol of run $S_i(j)$. We let $S_i^{(r)}$ denote the first r runs of S_i, i.e. $S_i^{(r)} = S_i(1)S_i(2)\ldots S_i(r)$.

We use $LCS(S_1, S_2, \ldots, S_d)$ to denote the length of LCS of strings S_1 to S_d. Let M be a matrix of *blocks*, such that $M[r_1, r_2, \ldots, r_d]$ contains the the value of $LCS(S_1^{(r_1)}, S_2^{(r_2)} \ldots, S_d^{(r_d)})$, where $r_i = 1, 2, \ldots, n_i$. We say that block $(r_1, r_2, \ldots, r_d)$ is *dark* if all the runs $S_1(r_1), S_2(r_2), \ldots,$ and $S_d(r_d)$ encode an identical symbol. Otherwise, we say that block $(r_1, r_2, \ldots, r_d)$ is *light*.

2.2 Properties for LCS of two RLE strings

Before describing the Lemmas of multiple strings, we first review the known properties of LCS between two RLE strings.

Lemma 1. *(see [7]) Let S_1 and S_2 be two strings and e^r be a run encoding symbol e. Let S_1e^r and S_2e^r denote the strings obtained by appending the run e^r to S_1 and S_2. We have that $LCS(S_1e^r, S_2e^r) = LCS(S_1, S_2) + r$.*

Lemma 2. *(see [7]) Let S_1 and S_2 be two strings and $e_1^{r_1}$ and $e_2^{r_2}$ be two runs encoding distinct symbols e_1 and e_2. Let $S_1e_1^{r_1}$ and $S_2e_2^{r_2}$ denote the strings obtained by appending the run $e_1^{r_1}$ to S_1 and the run $e_2^{r_2}$ to S_2. We have that $LCS(S_1e_1^{r_1}, S_2e_2^{r_2}) = \max\{LCS(S_1e_1^{r_1}, S_2), LCS(S_1, S_2e_2^{r_2})\}$.*

3 LCS of multiple RLE strings

We now describe several properties of the LCS of multiple RLE strings. Lemma 3 is dedicated to the case that all the input strings end with the same symbol, and Lemma 4 is dedicated to the case that the input strings not end with the same symbol. Lemma 5 is dedicated to the case when updating the value of a light block $(r_1, r_2, ...r_d)$.

Lemma 3. *Let $S_1, S_2, \ldots, S_d$ be d strings and e^r be a run. Let $S_1e^r, S_2e^r, \ldots, S_de^r$ denote the strings obtained by appending run e^r to $S_1, S_2, \ldots, S_d$, respectively. We have that $LCS(S_1e^r, S_2e^r, \ldots, S_de^r) = LCS(S_1, S_2, \ldots, S_d) + r$.*

Proof. If $LCS(S_1e, S_2e, \ldots, S_de) = LCS(S_1, S_2, \ldots, S_d) + 1$ holds, then the lemma can be obtained by recursively applying the above result r times. Therefore, in the following we aim to show that $LCS(S_1e, S_2e, \ldots, S_de) = LCS(S_1, S_2, \ldots, S_d) + 1$. Note that the LCS of S_1e, $S_2e, \ldots,$ and S_de either end with symbol e or not.

Case 1: if the LCS of S_1e, $S_2e, \ldots,$ and S_de end with symbol e, then wherever the ending symbol e appears in S_ie for $i = 1, \ldots, d$, we can always replace it by the last e of S_ie. Hence, we obtain that $LCS(S_1e, S_2e, \ldots, S_de) = LCS(S_1, S_2, \ldots, S_d) + 1$.

Case 2: if the LCS of S_1e, $S_2e, \ldots,$ and S_de does not end with symbol e, then we have that $LCS(S_1e, S_2e, \ldots, S_de) = LCS(S_1, S_2, \ldots, S_d)$. This leads to a contradiction, for we can obtain a longer common subsequence by appending the last common e to the LCS of S_1, $S_2, \ldots,$ and S_d.

□

Lemma 4. *Let $S_1, S_2, \ldots, S_d$ be d strings and let $e_1, e_2, \ldots, e_d$ be d symbols which are not all the same, i.e. $e_1 = e_2 = \ldots = e_d$ cannot hold. We have that for any $\sigma \in \{e_1, e_2, \ldots, e_d\}$,*

$$LCS(S_1e_1^{r_1}, S_2e_2^{r_2}, \ldots, S_de_d^{r_d}) = \max \begin{cases} LCS(\tau(S_1e_1^{r_1}, \sigma), \tau(S_2e_2^{r_2}, \sigma), \ldots, \tau(S_de_d^{r_d}, \sigma)), \\ LCS(\bar{\tau}(S_1e_1^{r_1}, \sigma), \bar{\tau}(S_2e_2^{r_2}, \sigma), \ldots, \bar{\tau}(S_de_d^{r_d}, \sigma)), \end{cases}$$

where $\tau(S_ie_i^{r_i}, \sigma) = \begin{cases} S_ie_i^{r_i}, & \text{if } e_i = \sigma; \\ S_i, & \text{otherwise,} \end{cases}$ *and*

$\bar{\tau}(S_ie_i^{r_i}, \sigma) = \begin{cases} S_i, & \text{if } e_i = \sigma; \\ S_ie_i^{r_i}, & \text{otherwise.} \end{cases}$

Proof. Given symbol $\sigma \in \{e_1, e_2, \ldots, e_d\}$, we know that the LCS of strings $S_1, S_2, \ldots, S_d$ either ends with σ or not. If the LCS ends with symbol σ, the last runs of the strings not encoding symbol σ need not to be considered, whereas the the last runs of the strings encoding symbol σ need to be considered. The function τ helps us to do this. On the other hand, if the LCS does not end with

symbol σ, the function $\bar{\tau}$ removes the last runs of the strings encoding symbol σ, while preserving the last runs of the strings not encoding symbol σ. □

Lemma 5. *Suppose that runs $S_1(r_1)$, $S_2(r_2)$, ... , $S_d(r_d)$ do not encode the same symbol, i.e. symbol$(S_1(r_1))$=symbol$(S_2(r_2))$=...=symbol$(S_d(r_d))$ cannot hold. We have that*

$$LCS(S_1^{(r_1)}, S_2^{(r_2)}, \ldots, S_d^{(r_d)}) = \max \begin{cases} LCS(S_1^{(r_1-1)}, S_2^{(r_2)}, \ldots, S_d^{(r_d)}), \\ LCS(S_1^{(r_1)}, S_2^{(r_2-1)}, \ldots, S_d^{(r_d)}), \\ \ldots \\ LCS(S_1^{(r_1)}, S_2^{(r_2)}, \ldots, S_d^{(r_d-1)}). \end{cases}$$

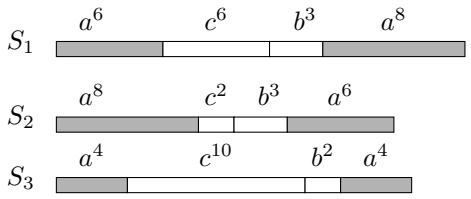
Proof. Suppose that the LCS of $S_1^{(r_1)}, S_2^{(r_2)}, \ldots,$ and $S_d^{(r_d)}$ end with symbol $\sigma \in \Sigma$. Since $symbol(S_1(r_1)) = symbol(S_2(r_2)) = \ldots = symbol(S_d(r_d))$ cannot hold, at least one of the runs $S_1(r_1), S_2(r_2), \ldots, S_d(r_d)$ does not encode σ. In other words, there is at least one string whose last run can be ignored. □

4 The first algorithm

Before describing the details of our algorithm, we first take an example to illustrate our basic idea.

4.1 The basic idea

Given three strings S_1, S_2, and S_3, of n_1, n_2, and n_3 runs respectively, we aim to compute $LCS(S_1^{(n_1)}, S_2^{(n_2)}, S_3^{(n_3)})$. In the following example, $n_1 = 4$, $n_2 = 4$, and $n_3 = 4$.



Our algorithm runs in a dynamic fashion. Suppose that all the values of $LCS(S_1^{(r_1)}, S_2^{(r_2)}, S_3^{(r_3)})$ for $r_1 \leq 3$, $r_2 \leq 3$, $r_3 \leq 3$ are already computed. We now show how to compute the value of $LCS(S_1^{(4)}, S_2^{(4)}, S_3^{(4)})$. We recursively apply Lemma 3 and Lemma 4 as follows:

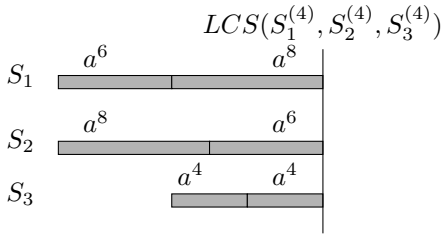
1. By Lemma 3, $LCS(S_1^{(4)}, S_2^{(4)}, S_3^{(4)}) = LCS(S_1^{(3)}a^4, S_2^{(3)}a^2, S_3^{(3)}) + 4$.
2. By Lemma 4, $LCS(S_1^{(3)}a^4, S_2^{(3)}a^2, S_3^{(3)}) = \max\{LCS(S_1^{(3)}, S_2^{(3)}, S_3^{(3)}), LCS(S_1^{(3)}a^4, S_2^{(3)}a^2, S_3^{(2)})\}$.
3. By Lemma 4, $LCS(S_1^{(3)}a^4, S_2^{(3)}a^2, S_3^{(2)}) = \max\{LCS(S_1^{(3)}, S_2^{(3)}, S_3^{(2)}), LCS(S_1^{(3)}a^4, S_2^{(3)}a^2, S_3^{(1)})\}$.
4. By Lemma 3, $LCS(S_1^{(3)}a^4, S_2^{(3)}a^2, S_3^{(1)}) = LCS(S_1^{(3)}a^2, S_2^{(3)}, a^2) + 2$.
5. By Lemma 4, $LCS(S_1^{(3)}a^2, S_2^{(3)}, a^2) = \max\{LCS(S_1^{(3)}, S_2^{(3)}, \epsilon), LCS(S_1^{(3)}a^2, S_2^{(2)}, a^2)\}$.
6. By Lemma 4, $LCS(S_1^{(3)}a^2, S_2^{(2)}, a^2) = \max\{LCS(S_1^{(3)}, S_2^{(2)}, \epsilon), LCS(S_1^{(3)}a^2, S_2^{(1)}, a^2)\}$.
7. By Lemma 3, $LCS(S_1^{(3)}a^2, S_2^{(1)}, a^2) = LCS(S_1^{(3)}, a^6, \epsilon) + 2$.

Combining the above equations, we can derive that:

$$LCS(S_1^{(4)}, S_2^{(4)}, S_3^{(4)}) = \max \begin{cases} LCS(S_1^{(3)}, S_2^{(3)}, S_3^{(3)}) + 4, \\ LCS(S_1^{(3)}, S_2^{(3)}, \epsilon) + 6, \\ LCS(S_1^{(3)}, a^6, \epsilon) + 8 \end{cases}$$

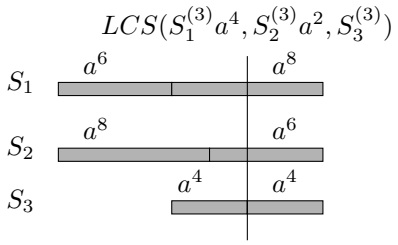
Since by assumption the value of $LCS(S_1^{(3)}, S_2^{(3)}, S_3^{(3)}) = 8$ is already computed, and obviously $LCS(S_1^{(3)}, S_2^{(3)}, \epsilon) = 0$ and $LCS(S_1^{(3)}, a^6, \epsilon) = 0$, we obtain that $LCS(S_1, S_2, S_3) = \max\{8 + 4, 0 + 6, 0 + 8\} = 12$.

Clearly, the time for the above procedure depends on the depth of the recursions. Figures (a) to (d) show that the depth of the recursions is no more than the total number of runs in S_1, S_2, and S_3. For clarity, we remove all the non-'a' runs of strings S_1, S_2, and S_3 in our illustrations.



By Lemma 3, $LCS(S_1^{(4)}, S_2^{(4)}, S_3^{(4)}) = LCS(S_1^{(3)}a^4, S_2^{(3)}a^2, S_3^{(3)})+4$.

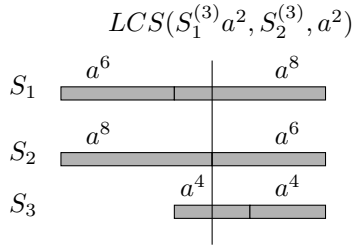
(a) Initially, the vertical line begins at the right boundaries runs $S_1(4)$, $S_2(4)$, and $S_3(4)$. When applying Lemma 3, the vertical line is moved to the left by four characters, stopping at the location shown in Figure (b).



By Lemma 4 (twice),
$LCS(S_1^{(3)}a^4, S_2^{(3)}a^2, S_3^{(3)}) = \max\{ LCS(S_1^{(3)}a^4, S_2^{(3)}a^2, S_3^{(1)}), LCS(S_1^{(3)}, S_2^{(3)}, S_3^{(3)})\}$.

By Lemma 3, $LCS(S_1^{(3)}a^4, S_2^{(3)}a^2, S_3^{(1)}) = LCS(S_1^{(3)}a^2, S_2^{(3)}, a^2)+2$.

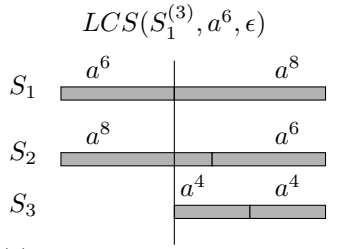
(b) When applying Lemma 4 and then Lemma 3, the vertical line is moved to the left by two characters, stopping at the location shown in Figure (c).



By Lemma 4 (twice),
$LCS(S_1^{(3)}a^2, S_2^{(3)}, a^2) = \max\{LCS(S_1^{(3)}a^2, S_2^{(1)}, a^2), LCS(S_1^{(3)}, S_2^{(3)}, \epsilon)\}$.

By Lemma 3, $LCS(S_1^{(3)}a^2, S_2^{(1)}, a^2) = LCS(S_1^{(3)}, a^6, \epsilon)+2$.

(c) When applying Lemma 4 and then Lemma 3, the vertical line is moved to the left by two characters, stopping at the location shown in Figure (d).



(d) The vertical line reaches the left end of string S_3 and the recursion is terminated.

4.2 The algorithm

Our algorithm for computing the LCS of multiple RLE strings is given in ALGORITHM 1. Recall that block $(r_1, r_2, \ldots, r_d)$ is a dark block when $S_1(r_1), S_2(r_2), \ldots,$ and $S_d(r_d)$ all encode the same symbol, and we call the common encoded symbol the *dark symbol* of block $(r_1, r_2, \ldots, r_d)$.

```
ALGORITHM 1. LCS(S_1, S_2, ..., S_d)
1  Matrix M is initialized as a zero matrix;
2  for r_1 = 1 to n_1
3  for r_2 = 1 to n_2
4  ...
5  for r_d = 1 to n_d
6  if (block (r_1, r_2, ..., r_d) is light)
7          M[r_1, r_2, ..., r_d] = max{
           M[r_1 - 1, r_2, ..., r_d],
           M[r_1, r_2 - 1, ..., r_d],
           ...
           M[r_1, r_2, ..., r_d - 1]};
8  else
9       Let σ be the dark symbol of
        block (r_1, r_2, ..., r_d);
10      for i = 1 to d do D[i] = |S_i(r_i)|;
11      matched = min{D[i] | i = 1, ..., d};
12      M[r_1, r_2, ..., r_d] =
        M[r_1 - 1, r_2 - 1, ..., r_d - 1] + matched;
13      DarkPath(r_1, ..., r_d, M, matched, D, σ);
```

In line 7, value of $M[r_1, r_2, \ldots, r_d]$ can be updated directly from Lemma 5. In line 10, $D[i]$ is initialized to the number of dark symbols σ in run $S_i(r_i)$. In line 11, variable *matched* indicates the number of matched dark symbols σ which runs $S_1(r_1)$ to $S_d(r_d)$ can contribute to the LCS.

```
DARKPATH(r_1, ..., r_d, M, matched, D, σ)
1  r'_1 = r_1, ..., r'_d = r_d;
2  while (every string contains at least
           one run encoding dark symbol σ
           which has not been exhausted)
3          for i = 1 to d
4                  if (D[i] == matched)
5                          update r'_i to indicate the
                           former run of S_i(r'_i) encoding
                           dark symbol σ and update
                           D[i] to D[i] + |S_i(r'_i)|;

6          matched = min{D[i] | i = 1, ..., d};
7          M[r_1, ..., r_d] = max{
           M[r'_1 - 1, ..., r'_d - 1] + matched,
           M[r_1, ..., r_d]};
```

In lines 3–5, procedure DARKPATH identifies those strings S_i whose run $S_i(r'_i)$ is exhausted. The procedure then updates r'_i to indicate the former run of $S_i(r'_i)$ encoding dark symbol σ and updates $D[i]$ to $D[i] + |S_i(r'_i)|$. Note that in line 5, $D[i]$ is updated to the number of dark symbols σ in substring $S_i(r'_i) \ldots S_i(r_i)$ of string S_i. In line 6, variable *matched* indicates the number of matched dark symbols σ which substrings $S_1(r'_1) \ldots S_1(r_1)$ to $S_d(r'_d) \ldots S_d(r_d)$ can contribute to the LCS. In line 7, the value of $M[r_1, r_2, \ldots, r_d]$ is updated based on Lemma 3 and Lemma 4.

The while-loop continues until there exists some string whose runs encoding dark symbol σ are completely exhausted. Since there are $\sum_{i=1}^{d} n_i$ runs in total, the while-loop is executed no more than $\sum_{i=1}^{d} n_i$ times. For each iteration of the while-loop, it takes $O(d)$ time. Hence, the procedure of DARKPATH spends $O(d \sum_{i=1}^{d} n_i)$ time.

Theorem 1. *Given d strings $S_1, \ldots, S_d$, compressed into $n_1, \ldots, n_d$ runs respectively,* ALGORITHM 1 *computes the LCS of $S_1, \ldots, S_d$ in $O(dnm)$ time, where $n = \sum_{i=1}^{d} n_i$ and $m = \prod_{i=1}^{d} n_i$.*

Proof. Each light block takes $O(d)$ time to update, while each dark block needs $O(dn)$ time to complete procedure DARKPATH. As there are m blocks in total, the total time required is $O(dnm)$. □

5 A faster algorithm

In this section, we improve upon the time of the first algorithm by exploiting the sparsity inherent in the LCS computation. Algorithm 2 will only store values of dark blocks. In addition, we propose Lemma 6 when we need a value of an light block by comparing the values of dark blocks.

Lemma 6. *Suppose that runs $S_1(r_1)$, $S_2(r_2)$, ... , $S_d(r_d)$ do not encode the same symbol, i.e. symbol$(S_1(r_1))$=symbol$(S_2(r_2))$=...=symbol$(S_d(r_d))$ cannot hold. We have that*

$$LCS(S_1^{(r_1)}, S_2^{(r_2)}, \ldots, S_d^{(r_d)}) = \max\{LCS(\acute{\tau}(S_1^{(r_1)}, \alpha), \acute{\tau}(S_2^{(r_2)}, \alpha), \ldots, \acute{\tau}(S_d^{(r_d)}, \alpha)) \mid \alpha \in \Sigma\},$$

where $\acute{\tau}(S_i^{(r_i)}, \alpha) = S_i^{(r_i)}$ if symbol$(S_i(r_i)) = \alpha$, and $\acute{\tau}(S_i^{(r_i)}, \alpha) = S_i^{(r'_i)}$ otherwise. Here, $S_i(r'_i)$ is the former run of $S_i(r_i)$ which encodes symbol α.

Proof. Suppose that the LCS of $S_1^{(r_1)}, S_2^{(r_2)}, \ldots,$ and $S_d^{(r_d)}$ ends with symbol $\alpha \in \Sigma$. Then, the last runs of the strings encoding α need to be considered, whereas the last runs of the strings not encoding α need not to be considered. The function $\acute{\tau}$ helps us to do this. □

```
ALGORITHM 2. LCS(S_1, ..., S_d)
1  Matrix M is initialized as a zero matrix;
2  for r_1 = 1 to n_d
3      σ = symbol(S_1(r_1))
4      for r_2 = S_2[σ].begin to S_2[σ].end
5      ...
6      for r_d = S_d[σ].begin to S_d[σ].end
7          for i = 1 to d do D[i] = |S_i(r_i)|;

8          matched = min{D[i] | i = 1, ..., d};
9          if (M[r_1 − 1, ..., r_d − 1] == 0)
10             prematched = max{
               M[pre(S_1, r_1 − 1, α), ..., pre(S_d, r_d − 1, α)]
                | α ∈ Σ };
11         else
               prematched = M[r_1 − 1, ..., r_d − 1];

12         M[r_1, ..., r_d] = prematched + matched;
13         DarkBlockOnly(r_1, ..., r_d, M, matched, D, σ);

14 if  (M[n_1, ..., n_d] == 0)
15     M[n_1, ..., n_d] = max{
       M[pre(S_1, n_1, α), ..., pre(S_d, n_d, α)] | α ∈ Σ }
```

In line 3, $symbol(S_1(r_1))$ returns the encoded symbol σ of run $S_1(r_1)$. In lines 4–6, array $S_i[\sigma]$ records the indices of runs encoding σ in S_i. For example, for string $S_i = a^5b^4c^7b^3a^4$, $S_i['a'] = \{1, 5\}$. Preprocessing $S_i[\sigma]$ for $\sigma \in \Sigma$ can be done in $O(\sum_{i=1}^{d} n_i)$ time. Then after line 6, block $(r_1, r_2, \ldots, r_d)$ is a dark block encoding symbol σ, i.e. σ is the dark symbol of block $(r_1, r_2, \ldots, r_d)$.

In line 7, $D[i]$ is initialized as the number of dark symbols σ in run $S_i(r_i)$. In line 8, variable *matched* indicates the number of matched dark symbols σ which runs $S_1(r_1)$ to $S_d(r_d)$ can contribute to the LCS. In lines 9–11, variable *prematched* indicates value of the LCS of $S_1^{(r_1-1)}$ to $S_d^{(r_d-1)}$. If value of $M[r_1 - 1, \ldots, r_d - 1]$ is zero, i.e. block $(r_1 - 1, \ldots, r_d - 1)$ is light, we compute its value directly from Lemma 6. Function $pre(S_i, r_i, \alpha)$ returns the index of the former run of $S_i(r_i)$ which encodes symbol α (inclusive of $S_i(r_i)$). For example, for string $S = a^5b^4c^7b^3a^4c^3$, $pre(S, 6, a) = 5$, $pre(S, 3, c) = 3$ and $pre(S, 1, b) = 0$. Preprocessing $pre(S_i, r_i, \alpha)$ for $\alpha \in \Sigma$ can be done in $O(|\Sigma| \sum_{i=1}^{d} n_i)$ time.

In lines 14–15, if value of $M[n_1, \ldots, n_d]$ is zero, i.e. block $(n_1, \ldots, n_d)$ is light, we compute its value directly from Lemma 6 for the LCS of S_1 to S_d.

```
DARKBOLCKONLY(r_1, ..., r_d, M, matched, D, σ)
1  r'_1 = r_1, ..., r'_d = r_d;
2  while (every string contains at least
           one run encoding dark symbol σ
           which has not been exhausted)
3          for i = 1 to d
4              if (D[i] == matched)
5                  update r'_i to indicate the
                   former run of S_i(r'_i) encoding
                   dark symbol σ and update
                   D[i] to D[i] + |S_i(r'_i)|;

6          matched = min{D[i] | i = 1, ..., d};
7          if (M[r'_1 − 1, ..., r'_d − 1] == 0)
8              prematched = max{
               M[pre(S_1, r'_1 − 1, α), ..., pre(S_d, r'_d − 1, α)] | α ∈ Σ }
9          else
               prematched = M[r'_1 − 1, ..., r'_d − 1];

10         M[r_1, ..., r_d] = max{
           prematched + matched, M[r_1, ..., r_d]}
```

In lines 3–5, procedure DARKBOLCKONLY identifies those strings S_i whose run $S_i(r'_i)$ is exhausted. The procedure then updates r'_i to indicate the former run of $S_i(r'_i)$ encoding dark symbol σ and updates $D[i]$ to $D[i] + |S_i(r'_i)|$. Note that in line 5, $D[i]$ is updated to the number of dark symbols σ in substring $S_i(r'_i) \ldots S_i(r_i)$ of string S_i. In line 6, variable *matched* indicates the number of matched dark symbols σ which substrings $S_1(r'_1) \ldots S_1(r_1)$ to $S_d(r'_d) \ldots S_d(r_d)$ can contribute to the LCS. In lines 7–9, variable *prematched* indicates value of the LCS of $S_1^{(r'_1-1)}$ to $S_d^{(r'_d-1)}$. If value of $M[r'_1 - 1, \ldots, r'_d - 1]$ is zero, i.e. block $(r'_1 - 1, \ldots, r'_d - 1)$ is light, we compute its value directly from Lemma 6. It can be done in $O(d|\Sigma|)$ time. In line 10, the value of $M[r_1, r_2, \ldots, r_d]$ is updated based on Lemma 3 and Lemma 4.

The while-loop continues until there exists some string whose runs encoding dark symbol σ are completely exhausted. Since there $\sum_{i=1}^{d} n_i$ runs in total, the while-loop is executed no more than $\sum_{i=1}^{d} n_i$ times. For each iteration of the while-loop, it takes $O(d|\Sigma|)$ time. Hence, the procedure of DARKBOLCKONLY spends $O(d|\Sigma| \sum_{i=1}^{d} n_i)$ time.

Theorem 2. ALGORITHM 2 *takes* $O(dn\rho|\Sigma|)$ *times, where* ρ *denotes the number of dark blocks plus one and* $n = \sum_{i=1}^{d} n_i$ *denotes the total number of runs of strings* $S_1, S_2, \ldots, S_d$.

Proof. The preprocessing for $S_i[\sigma]$, $\sigma \in \Sigma$, and $pre(S_i, r_i, \alpha)$, $\alpha \in \Sigma$, can be done in $O(n)$ and

$O(n|\Sigma|)$ time, respectively. The light blocks need not to be updated, while the dark blocks take $O(dn|\Sigma|)$ time to run the procedure of DARK-BOLCKONLY. Hence, the total time required is $O(n)+O(n|\Sigma|)+ O(dn|\Sigma|) \times (\rho - 1)$ time. The time complexity thus follows. □

Note that ALGORITHM 2 limits the LCS computation to those *dark blocks*, whose number is specified by parameter ρ. Since $\rho = O(m)$, the time of ALGORITHM 2 is never worse than that of ALGORITHM 1 for constant-size alphabets. In other words, for cases where dark blocks are sparse, ALGORITHM 2 is preferable over ALGORITHM 1 for small alphabets. On the other hand, for random input strings the probability of the appearance of a dark block is $\frac{1}{|\Sigma|^{d-1}}$, which implies that $\rho = O(\frac{1}{|\Sigma|^{d-1}} * m)$. Hence, we obtain that in average case the time of ALGORITHM 2 is $O(dn\rho|\Sigma|) = \frac{1}{|\Sigma|^{d-2}} * O(dnm)$, which is superior to that of ALGORITHM 1.

6 Concluding Remarks

In this paper, we study the problem of computing LCS of multiple RLE strings. Two algorithms are presented, both can directly cope with the RLE strings without resorting to decompression. The first algorithm works in a dynamic fashion, recursively applying the lemmas described in Section 3. The second algorithm improves upon the time by exploiting the sparsity inherent in the LCS computation. It remains open whether the time can be further reduced by utilizing more efficient data structures and algorithmic techniques.

7 Acknowledgements

The authors would like to thank Kun-Mao Chao for his constructive comments.

References

[1] Amihood Amir and Gary Benson: Efficient Two-Dimensional Compressed Matching. *DCC*: 279–288, 1992.

[2] Hsing-Yen Ann, Chang-Biau Yang, Chiou-Ting Tseng, Chiou-Yi Hor: A Fast and Simple Algorithm for Computing the Longest Common Subsequence of Run-Length Encoded Strings. *Information Processing Letters* 108(6): 360–364, 2008

[3] Alberto Apostolico, Gad M. Landau, and Steven Skiena: Matching for Run-Length Encoded Strings. *Journal of Complexity* 15(1):4–16, 1999.

[4] Horst Bunke and Janos Csirik: An Improved Algorithm for Computing the Edit Distance of Run-Length Coded Strings. *Information Processing Letters* 54(2): 93–96, 1995.

[5] Kuan-Yu Chen, Kun-Mao Chao: A Fully Compressed Algorithm for Computing the Edit Distance of Run-Length Encoded Strings. *Algorithmica*, to appear.

[6] Maxime Crochemore, Gad M. Landau, Michal Ziv-Ukelson: A Subquadratic Sequence Alignment Algorithm for Unrestricted Scoring Matrices. *SIAM Journal on Computing* 32(6): 1654–1673, 2003.

[7] Valerio Freschi, Alessandro Bogliolo: Longest Common Subsequence between Run-Length-Encoded Strings: a New Algorithm with Improved Parallelism. *Information Processing Letters* 90(4): 167–173, 2004.

[8] Guan-Shieng Huang, Jia Jie Liu, Yue-Li Wang: Sequence Alignment Algorithms for Run-Length-Encoded Strings. *COCOON*, 319–330, 2008.

[9] Veli Mäkinen, Esko Ukkonen, Gonzalo Navarro: Approximate Matching of Run-Length Compressed Strings. *Algorithmica* 35(4): 347–369, 2003.